

Министерство образования и науки Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий радиоэлектроники
Кафедра информационных систем

КУРСОВАЯ РАБОТА

по курсу Прикладная разработка на Java
на тему: Реализация HTTP-сервера по проведению опросов

Руководитель

(уч. степень, звание)

Метелкин А.С.

(фамилия, инициалы)

(подпись)

(дата)

Члены комиссии

Студент ИС-123

(группа)

Шпидонов Н.А.

(фамилия, инициалы)

(подпись)

(Ф.И.О.)

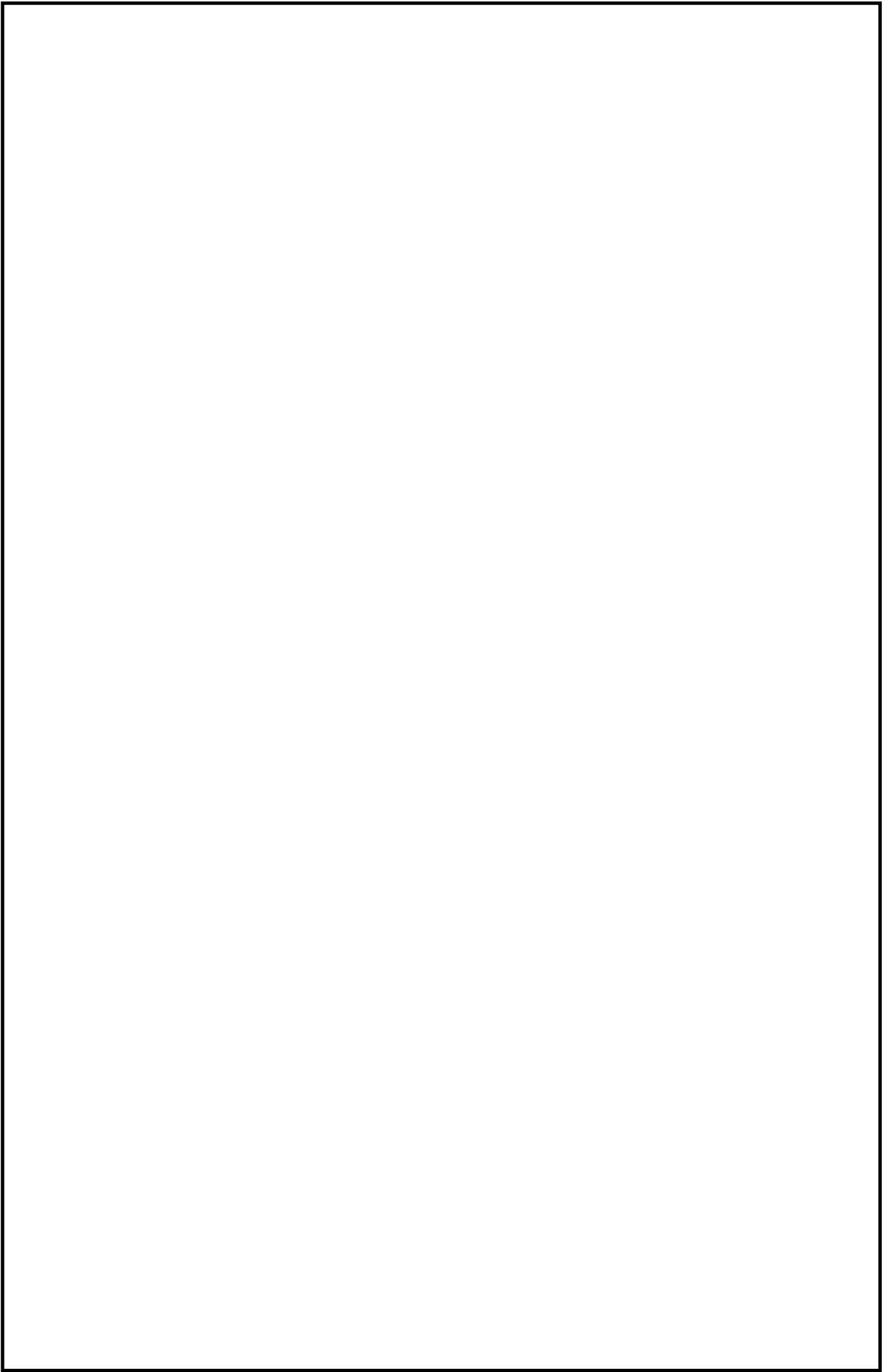
(подпись)

(Ф.И.О.)

(подпись)

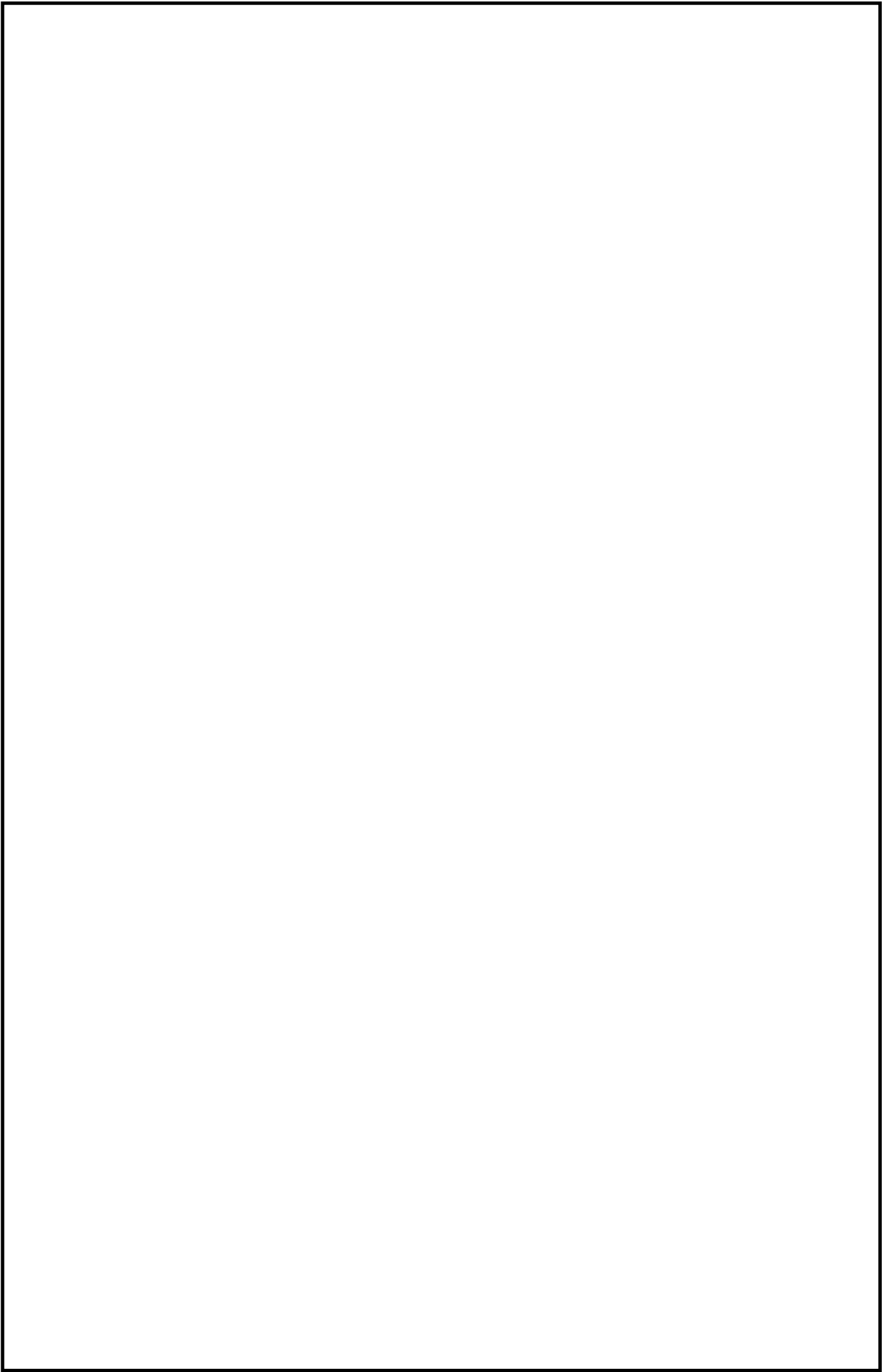
(дата)

Муром 2026



В курсовой работе отражена разработка веб-сервиса для создания и проведения опросов на языке Java с использованием фреймворка Spring Boot. В ходе работы спроектирована структура реляционной базы данных (Firebird), обеспечивающая хранение данных о пользователях, формах опросов, вопросах различных типов и полученных ответах. Реализован интуитивно понятный пользовательский интерфейс на базе шаблонизатора Thymeleaf и обеспечена многоуровневая система безопасности с помощью Spring Security.

Особое внимание в работе уделено модулю аналитики: разработана система сбора результатов и генерации отчетов в нескольких форматах (HTML, XML, TXT), что позволяет гибко обрабатывать накопленные данные. Корректность работы бизнес-логики подтверждена в ходе модульного тестирования. Разработанный сервис представляет собой готовый инструмент для автоматизации сбора обратной связи и может быть использован в исследовательских, маркетинговых или образовательных целях.



Содержание

Введение	5
1. Анализ технического задания	6
1.1.Цель и задачи разработки.....	6
1.2.Архитектура системы	6
1.3.Функциональные требования к системе	9
1.4.Управление опросами	12
2. Проектирование программы	14
2.1.Проектирование архитектуры и структуры данных системы	14
2.2.Проектирование информационной модели и базы данных	18
3. Разработка приложения	22
3.1.Выбор и обоснование технологического стека.....	22
3.2.Реализация системы типов вопросов	24
3.3.Реализация работы с мультимедиа-контентом	25
3.4.Реализация управления жизненным циклом опроса	26
3.5.Реализация интерфейса прохождения и сбора данных.....	27
3.6.Реализация аналитики и экспорта результатов.....	28
3.7.Архитектурные особенности реализации.....	30
4. Тестирование.....	32
Заключение.....	37
Библиография.....	39

					МИВУ.09.03.02 025 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Шпидонов Н.А			Курсовой проект по теме: «Реализация HTTP-сервера по проведению опросов»		Лит.	Лист
Провер.		Метелкин А.С.						4
Реценз.							ИС-123	
Н. Контр.								
Утверд.								

функциональная страница

					МИВУ.09.03.02 025 ПЗ									
Изм.	Лист	№ докум.	Подпись	Дата	Курсовой проект по теме: «Реализация HTTP-сервера по проведению опросов»				Лит.		Лист	Листов		
Разраб.		Шпидонов Н.А										4		
Провер.		Метелкин А.С.							ИС-123					
Реценз.														
Н. Контр.														
Утверд.														

функциональная страница

					МИВУ.09.03.02 025 ПЗ	Лист
						4
Изм.	Лист	№ докум.	Подпись	Дата		

Введение

С развитием информационных технологий и повсеместным распространением интернета всё больше растет потребность в инструментах для сбора и анализа данных. Современные веб-технологии позволяют создавать интерактивные сервисы, которые делают процесс сбора информации удобным и эффективным. Эти методы нашли широкое применение в различных областях, от маркетинговых исследований до академических изысканий.

Тема курсовой работы – разработка сервиса для проведения опросов. Объектом исследования является процесс создания, проведения и анализа результатов онлайн-опросов. Предметом исследования является разработанный веб-сервис, реализующий данный функционал.

Цель курсовой работы – спроектировать и разработать многофункциональный веб-сервис для проведения опросов. В рамках работы будет разработана система, которая позволяет пользователям создавать опросы, проходить их, а администраторам - анализировать собранные данные.

Исходя из поставленной цели, определены следующие задачи:

Изучение литературных источников по теме проектирования баз данных и разработки веб-приложений.

Разработка структуры базы данных для хранения информации об опросах, вопросах, ответах и пользователях.

Реализация серверной части приложения (backend), отвечающей за бизнес-логику: управление опросами, обработку ответов и формирование отчетов.

Создание пользовательского интерфейса (frontend) для создания и прохождения опросов, а также для просмотра статистики и результатов.

Тестирование и отладка разработанного сервиса.

Результатом работы будет создан веб-сервис, который позволяет создавать и проводить опросы, собирать ответы и анализировать их с помощью инструментов статистики и визуализации.

					МИВУ.09.03.02 025 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

1. Анализ технического задания

1.1. Цель и задачи разработки

Целью данной курсовой работы является проектирование и разработка веб-приложения для создания, прохождения и администрирования пользовательских опросов.

Для достижения поставленной цели необходимо выполнить декомпозицию и решить следующие задачи:

1. Провести анализ существующих решений (аналогов) на рынке.
2. Выбрать оптимальный стек технологий и инструментов для разработки.
3. Спроектировать структуру базы данных для хранения форм, вопросов и ответов.
4. Разработать серверную часть (backend) с использованием паттернов проектирования и реализовать бизнес-логику.
5. Создать пользовательский интерфейс (frontend) для трех основных ролей (администратор, создатель опроса, респондент).
6. Реализовать функционал экспорта результатов (генерация отчетов).
7. Провести модульное (Unit Testing) и ручное тестирование готового приложения.

1.2. Сравнительный анализ существующих решений

В ходе исследования были рассмотрены популярные сервисы для создания опросов. Основной акцент делался на логику работы и предоставляемый функционал. Анализ аналогов и обоснование выбора технологий представлены в таблицах 1 и 2.

					МИВУ.09.03.02 025 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

Google Формы — первый и основной аналог, на примере которого была спроектирована базовая логика разрабатываемого веб-приложения (создание вопросов различных типов, простота прохождения). Интерфейс представлен на рисунке 1.

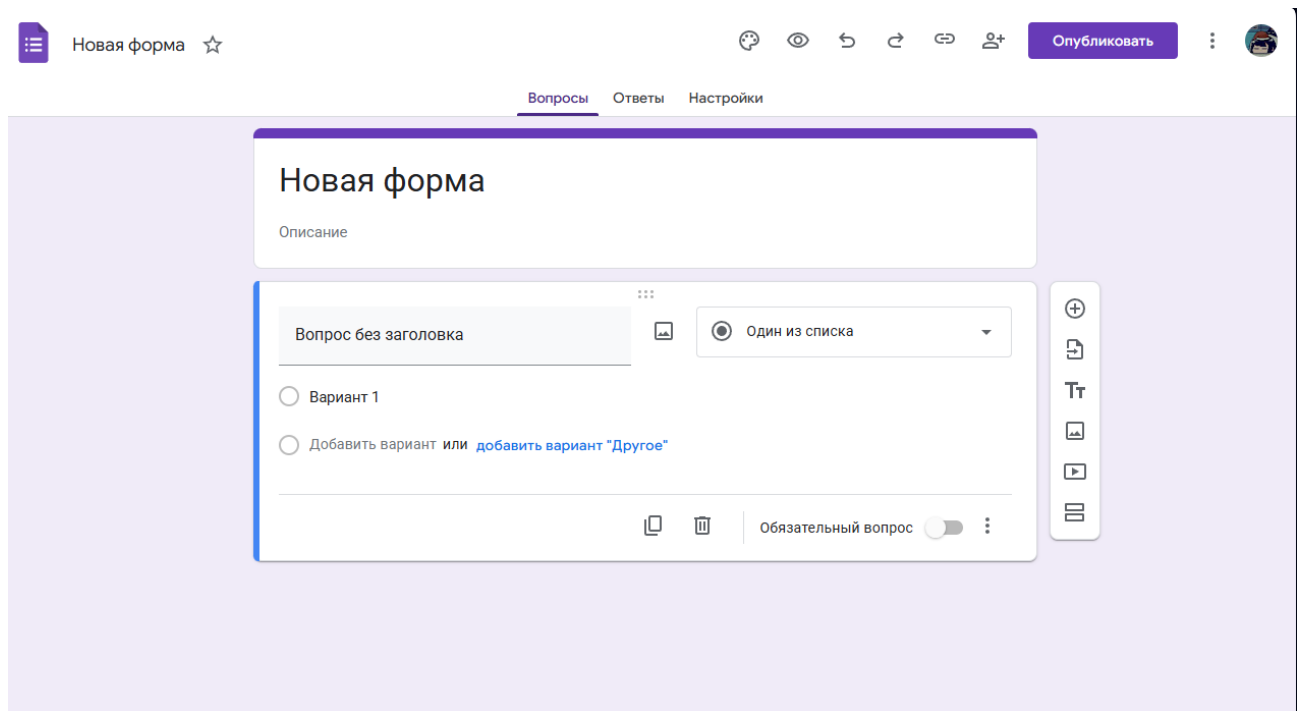


Рисунок 1 – Интерфейс google формы

Yandex Формы — второй рассмотренный аналог, обладающий расширенным функционалом, включая смену тем оформления, интеграцию с почтой и множеством дополнительных настроек. Интерфейс представлен на рисунке 2.

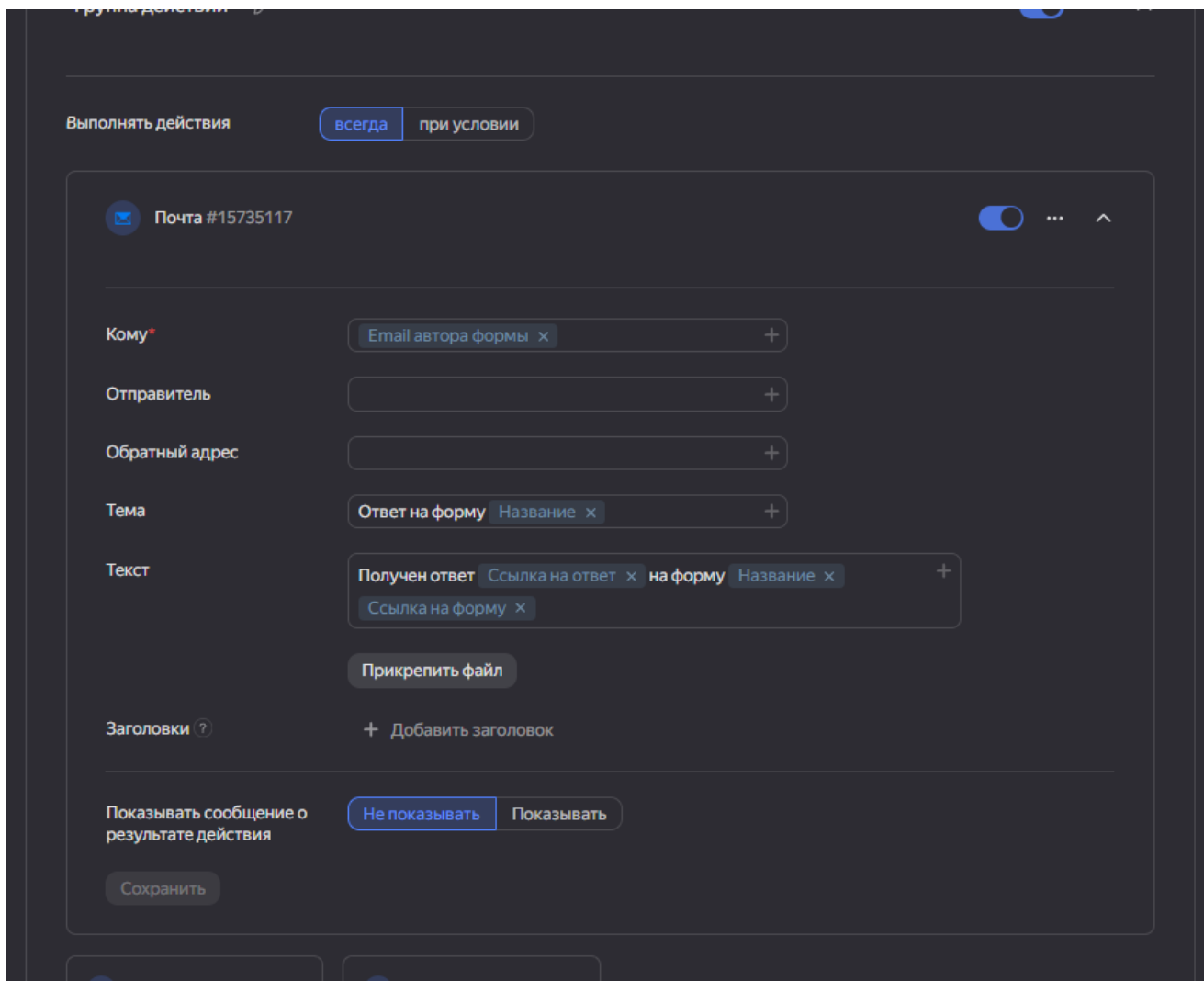


Рисунок 2 – Интерфейс yandex формы

Таблица 1 — Сравнительный анализ аналогов и разрабатываемого решения

Критерий сравнения	Google Формы	Yandex Формы	Разрабатываемое приложение
Среда развертывания	Облако (SaaS)	Облако (SaaS)	Локальный сервер
Хранение данных	Зарубежные серверы	Серверы РФ	Локальная БД (Firebird SQL)
Стоимость использования	Бесплатно (базово)	Бесплатно (базово)	Бесплатно
Возможность доработки	Отсутствует	Отсутствует	Полная (доступ к исходному коду)
Интеграция со своей БД	Только через API/Скрипты	Только через API	Прямая работа (JPA/Hibernate)

Таблица 2 — Обоснование выбора технологий

Технология	Назначение	Аналоги	Почему выбрана
Java 17	Основной язык программирования	C#, Python, PHP	Строгая типизация, объектно-ориентированный подход, высокий уровень безопасности и кроссплатформенность.
Spring Boot 3	Фреймворк для Backend-разработки	Django, Node.js (Express)	Стандарт в Enterprise-разработке. Предоставляет готовые модули (Spring Data JPA, Spring Security) "из коробки", ускоряя процесс.

Продолжение таблицы 2 — Обоснование выбора технологий

Технология	Назначение	Аналоги	Почему выбрана
Firebird SQL	Система управления базами данных (СУБД)	MySQL, PostgreSQL	Легковесная реляционная БД, проста в настройке и администрировании. Отлично подходит для учебных и локальных проектов.
Thymeleaf	Шаблонизатор (Frontend)	React, Angular	Рендеринг HTML на стороне сервера (SSR). Позволяет тесно связать Java-код и интерфейс без написания сложного API для отдельного фронтенда.
HTML/CSS/JS	Оформление и логика интерфейса	-	Стандартные технологии веба. Обеспечивают адаптивность.

Таким образом, на основе проведенного сравнительного анализа существующих решений и обоснования выбора технологий можно сделать вывод, что разработка собственного веб-сервиса на стеке Java 17 и Spring Boot 3 является наиболее целесообразной. Это позволит реализовать гибкую систему управления опросами с использованием локальной базы данных Firebird SQL, обеспечивая полную независимость от сторонних облачных сервисов, возможность глубокой кастомизации функционала и прямой доступ к данным через ORM Hibernate.

1.3. Архитектура системы

Веб-сервисы для сбора данных - это программные системы, которые позволяют организовать сбор, хранение и анализ информации от пользователей через интернет. Этот процесс происходит за счет интерактивного взаимодействия пользователя с интерфейсом приложения, который, в свою очередь, обменивается данными с сервером и базой данных. Примером такого сервиса может служить система для проведения онлайн-опросов.

Сервис опросов является важным инструментом, который описывается законами клиент-серверной архитектуры и принципами проектирования баз данных. Один из классических подходов к построению таких систем - это архитектурный паттерн «Модель-Представление-Контроллер» (MVC), который позволяет логически разделить компоненты системы:

Модель (Model): Представляет данные и бизнес-логику. Включает в себя классы, описывающие сущности (Опрос, Вопрос, Пользователь), и сервисы для взаимодействия с базой данных.

Представление (View): Отвечает за пользовательский интерфейс. В проекте используются шаблоны Thymeleaf (HTML/CSS), которые динамически отображают данные, полученные от контроллера, и предоставляют пользователю элементы для взаимодействия.

Контроллер (Controller): Принимает HTTP-запросы от клиента, обрабатывает их, обращается к модели за данными или для выполнения операций, и возвращает представление пользователю.

Эта архитектура обеспечивает гибкость, масштабируемость и упрощает поддержку кода. Общая архитектурная схема представлена на рисунке 3.

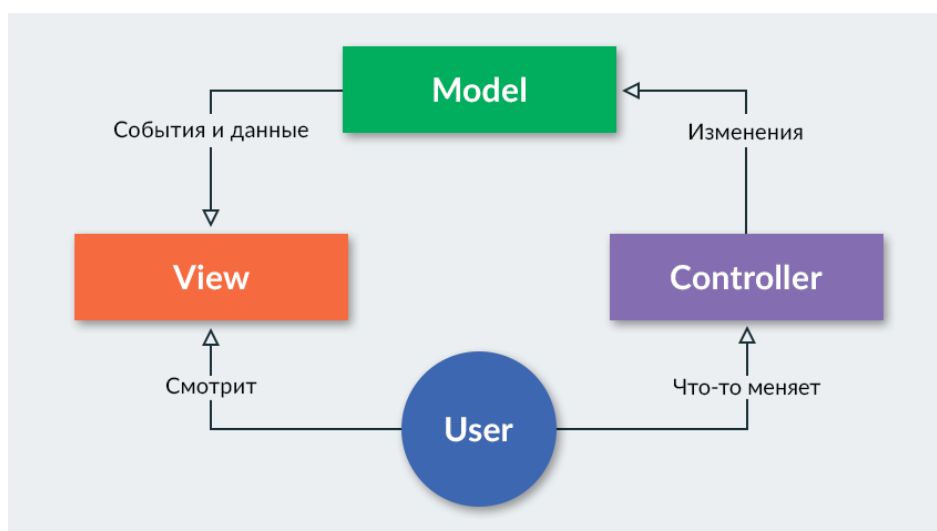


Рисунок 3 – Общая архитектурная схема приложения MVC

Жизненный цикл данных в системе опросов можно описать следующими этапами:

- Этап проектирования и создания (Backend и Frontend разработка) - процесс, в ходе которого разработчик формирует структуру базы данных, реализует бизнес-логику на сервере для управления опросами, пользователями и ответами, а также создает пользовательский интерфейс. С каждым шагом система становится все более функциональной.

– Этап сбора данных (Прохождение опроса) - процесс, в ходе которого конечные пользователи отвечают на вопросы опубликованного опроса. Система фиксирует их ответы и сохраняет в базе данных для последующей обработки.

– Этап анализа данных (Агрегация и отчетность) - процесс обработки и агрегации собранных ответов, при котором система преобразует разрозненные данные в структурированные отчеты, таблицы и диаграммы, предоставляя создателю опроса полезную статистику.

1.3.1 Функциональные требования к системе

Функциональные компоненты системы можно разделить на несколько типов в зависимости от их роли в архитектуре приложения.

1.3.2 Подсистема хранения данных (База данных):

В этой подсистеме происходит непосредственное хранение всей информации. Для проекта была спроектирована реляционная схема данных, включающая таблицы для пользователей, опросов, вопросов, вариантов ответов и самих ответов.

Для оптимизации запросов и формирования отчетов могут использоваться хранимые процедуры.

1.3.3 Подсистема бизнес-логики (Backend):

На этом уровне происходит вся обработка данных: проверка прав доступа пользователей (Администратор/Пользователь), применение фильтров при поиске опросов, вызов процедур из базы данных, расчет статистики. Этот компонент управляет всем жизненным циклом данных в системе.

1.3.4 Подсистема представления (Frontend):

В этой подсистеме пользователю предоставляется интерфейс для взаимодействия с системой. Это включает формы для создания опросов, страницы для их прохождения и панели для просмотра результатов. Для наглядной визуализации данных могут использоваться JavaScript-библиотеки, такие как Chart.js.

1.3.5 Регистрация и аутентификация

					МИВУ.09.03.02 025 ПЗ	Лист
						11
Изм.	Лист	№ докум.	Подпись	Дата		

Система должна предоставлять возможность для создания новых учетных записей. Процесс регистрации должен запрашивать у пользователя логин и пароль. После успешной регистрации пользователь должен иметь возможность войти в систему (аутентифицироваться) с использованием своих учетных данных. Пароли в базе данных должны храниться в хешированном виде для обеспечения безопасности.

1.3.6 Ролевая модель

В системе реализована ролевая модель для разграничения прав доступа.

Права Администратора:

- Просмотр, редактирование и удаление любых опросов в системе.
- Просмотр статистики по любому опросу.
- Права Пользователя:
- Создание, редактирование и удаление только собственных опросов.
- Просмотр списка всех опубликованных опросов.
- Прохождение опросов.
- Просмотр статистики только по собственным опросам.

1.4. Управление опросами

Создание, редактирование и удаление опроса являются основными функциями, однако присутствуют и другие.

Авторизованные пользователи должны иметь возможность создавать опросы через специальную форму, где указываются название и описание. Пользователь, создавший опрос, является его владельцем и может в дальнейшем его редактировать или удалить.

1.4.1 Просмотр и фильтрация опросов

На странице публичных опросов приложения должен отображаться список всех опубликованных опросов. Для удобства навигации должна быть реализована

					МИВУ.09.03.02 025 ПЗ	Лист
						12
Изм.	Лист	№ докум.	Подпись	Дата		

функция поиска по названию. Это позволяет пользователям легко находить нужные опросы среди множества доступных.

1.4.2 Поддерживаемые типы вопросов

- Текстовый ответ (одна строка/абзац): Поле для ввода произвольного текста.
- Один из списка: Набор вариантов с возможностью выбора только одного (radio buttons).
- Несколько из списка: Набор вариантов с возможностью выбора нескольких (checkboxes).
- Шкала: Линейная шкала (например, от 1 до 10) для оценки степени согласия.
- Сетка (множественный выбор): Таблица для оценки нескольких параметров по одной шкале.
- Загрузка файла (изображения): Возможность прикрепить к ответу графический файл.

В качестве основного подхода для реализации был выбран комплексный подход, сочетающий все три компонента в рамках архитектуры Spring Boot MVC.

					МИВУ.09.03.02 025 ПЗ	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		

2. Проектирование программы

2.1. Проектирование архитектуры и структуры данных системы

Для разработки многофункционального веб-сервиса по проведению опросов на начальном этапе была проведена детальная формализация основных задач проекта. Программа должна была решать комплекс задач: от динамического построения структуры анкет администратором до обеспечения защищенного доступа респондентов и последующей глубокой агрегации данных. Ключевым требованием стала необходимость создания гибкой системы, способной обрабатывать различные типы вопросов и генерировать отчеты в нескольких форматах. Дополнительно требовалось обеспечить расширяемость архитектуры, чтобы в будущем можно было интегрировать новые форматы экспорта или способы визуализации без изменения существующего ядра системы.

После определения функциональных требований была спроектирована общая архитектура программы. Для обеспечения надежности и упрощения дальнейшей поддержки проекта было принято решение разделить систему на несколько независимых уровней в рамках многослойной архитектуры (Layered Architecture). В результате проектирования были выделены следующие модули: уровень постоянного хранения данных (Persistence Layer), уровень бизнес-логики (Service Layer), уровень управления доступом (Security Layer) и уровень взаимодействия с пользователем (Presentation Layer). Такое разделение позволило изолировать логику обработки данных от способов их отображения, что значительно повысило чистоту архитектуры.

Следующим этапом стало проектирование подсистемы работы с данными. Поскольку опросы имеют иерархическую структуру (опрос включает вопросы, которые, в свою очередь, могут содержать варианты ответов), было важно спроектировать схему базы данных, минимизирующую избыточность. Для реализации этого уровня в проекте было запланировано использование объектно-

					МИВУ.09.03.02 025 ПЗ	Лист
						14
Изм.	Лист	№ докум.	Подпись	Дата		

реляционного отображения (ORM) на базе Hibernate. Проектирование включало описание сущностей User, Form, Question, Option и Answer. Для унификации доступа к этим данным на этапе проектирования была заложена система интерфейсов-репозиториев, наследуемых от базового интерфейса JpaRepository. Это решение позволило определить стандартный набор операций (поиск, сохранение, удаление) для всех сущностей еще до реализации конкретных запросов.

После проектирования уровня данных началась разработка архитектуры подсистемы бизнес-логики. Основной задачей на этом этапе стало отделение логики управления состоянием опроса от механизмов его хранения. Было принято решение создать специализированные сервисные интерфейсы, такие как FormService и ResponseService. Например, при проектировании FormService была заложена логика «сборки» сложной анкеты, где сервис выступает координатором между репозиториями вопросов и настроек. Это позволило сделать бизнес-правила (например, проверку обязательности ответа или ограничение по времени прохождения) централизованными и независимыми от контроллеров.

Особое внимание в процессе проектирования было уделено подсистеме аналитики и отчетности. Поскольку программа должна была поддерживать экспорт данных в форматах HTML, XML и TXT, требовался гибкий механизм переключения логики формирования файлов. Для решения этой задачи был спроектирован интерфейс ReportGenerator, определяющий общий контракт для всех модулей генерации. Использование паттерна проектирования «Стратегия» позволило отделить логику формирования документа от процесса сбора данных. Согласно архитектурному плану, основная программа должна взаимодействовать только с интерфейсом ReportGenerator, не зная о том, какая именно реализация (HTML или XML) используется в данный момент. Это обеспечило соблюдение принципа открытости/закрытости (Open/Closed Principle) из SOLID.

Следующим важным этапом стало проектирование системы безопасности и разграничения прав доступа. Для защиты персональных данных пользователей и

					МИВУ.09.03.02 025 ПЗ	Лист
						15
Изм.	Лист	№ докум.	Подпись	Дата		

результатов опросов было решено внедрить модуль на базе Spring Security. Архитектурный план включал гибкую модель контроля доступа:

Уровень авторизованного пользователя: спроектирована возможность создания собственных опросов, их редактирования, публикации и удаления. Реализована логика проверки владения объектом: пользователь имеет доступ к управлению формой только в том случае, если его идентификатор совпадает с полем `owner_id` в базе данных.

Уровень администратора: спроектированы расширенные полномочия (роль ADMIN), позволяющие просматривать и модерировать любые формы в системе, независимо от авторства.

Уровень гостя: доступ только к публичным страницам (регистрация, вход, прохождение опубликованных опросов).

Для реализации этого плана было выбрано использование интерфейса `UserDetailsService`. Проектирование взаимодействия включало создание защищенного контекста, где информация о текущем пользователе доступна на всех уровнях приложения, что позволяет динамически изменять интерфейс (например, скрывать кнопки «Редактировать» для чужих опросов). Для защиты конфиденциальной информации было запланировано использование криптографического хеширования паролей по стандарту BCrypt.

Затем была спроектирована подсистема пользовательского интерфейса. Основной задачей стало создание динамических представлений, которые могли бы адаптироваться под 8 различных типов вопросов (от простого текстового ввода до сложных сеток и шкал). Было принято решение использовать шаблонизатор Thymeleaf, который позволяет проектировать интерфейсы в виде чистых HTML-документов с внедрением логики на стороне сервера. Проектирование визуализации результатов включало интеграцию JavaScript-библиотеки Chart.js. Планировалось, что сервер будет передавать агрегированную статистику в формате JSON, а интерфейсный модуль будет преобразовывать её в интерактивные диаграммы.

					МИВУ.09.03.02 025 ПЗ	Лист
						16
Изм.	Лист	№ докум.	Подпись	Дата		

После проектирования всех отдельных компонентов была детально проработана логика их взаимодействия. Сценарий работы системы был определен следующим образом: пользователь инициирует действие через интерфейс, запрос попадает в соответствующий контроллер, который обращается к сервисному слою. Сервис, в свою очередь, взаимодействует с репозиториями для получения данных и, при необходимости, вызывает модуль генерации отчетов. После завершения обработки результат возвращается пользователю в виде динамически сформированной страницы или файла для загрузки.

При проектировании также особое внимание уделялось расширяемости. Благодаря использованию системы интерфейсов, добавление нового типа вопроса или нового формата экспорта не требует изменения существующей логики системы. Достаточно будет создать новый класс, реализовать соответствующий интерфейс и зарегистрировать его в контексте приложения.

Таким образом, в основе архитектуры проектируемой программы лежит система интерфейсов и слоев, через которые взаимодействуют все компоненты:

- Интерфейсы Repositories: отвечают за унифицированный доступ к реляционным данным;
- Интерфейс ReportGenerator: определяет контракт для формирования различных форматов аналитических отчетов;
- Интерфейс UserDetailsService: обеспечивает интеграцию системы безопасности с хранилищем учетных данных;
- Сервисные слои: инкапсулируют бизнес-логику управления жизненным циклом опросов.

Благодаря такой модульной архитектуре каждый компонент выполняет строго ограниченную задачу, что позволяет системе быть гибкой, отказоустойчивой и готовой к дальнейшему масштабированию.

2.1.1 Схема взаимодействия интерфейсов

Пользовательский интерфейс системы спроектирован с учетом ролевой модели. Гость системы имеет доступ к главной странице с публичными опросами и странице прохождения анкетирования. После успешной авторизации

пользователь перенаправляется в личный кабинет, где ему предоставляется доступ к управлению жизненным циклом опроса: создание структуры, добавление вопросов различных типов, публикация и последующий анализ собранных данных на странице результатов. Схема взаимодействия основных экранных форм представлена на рисунке 4.



Рисунок 4 - Схема взаимодействия интерфейсов

2.2. Проектирование информационной модели и базы данных

Важнейшим этапом проектирования системы является разработка информационной модели, которая должна обеспечивать целостность, непротиворечивость и минимальную избыточность хранимых данных. Поскольку разрабатываемый сервис предполагает работу со сложными динамическими структурами (различные типы вопросов, неограниченное количество ответов), в качестве модели хранения была выбрана реляционная модель.

На этапе проектирования была разработана логическая схема базы данных, включающая семь основных сущностей, взаимодействующих между собой:

1. Сущность «Пользователь» (Users)

Данная сущность является базовой для обеспечения безопасности и авторства. При проектировании были выделены атрибуты: уникальный идентификатор, логин, адрес электронной почты, хэшированный пароль и флаг административных прав.

Спроектировано ограничение уникальности (UNIQUE) для полей логина и почты, что предотвращает создание дублирующих учетных записей на уровне ядра БД.

2. Сущность «Форма опроса» (Forms)

Центральный объект системы. Опрос содержит метаданные: заголовок, описание и статус публикации. При проектировании заложена связь «многие-к-одному» с сущностью «Пользователь», что позволяет системе определять владельца формы для проверки прав на редактирование. Также предусмотрено поле для хранения даты создания, что необходимо для последующей реализации функций сортировки и фильтрации.

3. Сущность «Вопрос» (Questions)

Сущность описывает структуру анкеты. Ключевым атрибутом является «Тип вопроса», который определяет, каким образом данные будут отображаться и валидироваться. Спроектирована связь «многие-к-одному» с «Формой». Особенностью проектирования является сортировка вопросов по идентификатору создания (ID), что обеспечивает сохранение хронологической последовательности добавления.

4. Сущности «Вариант ответа» (Options) и «Настройки вопроса» (Question_Settings)

Для закрытых вопросов (выбор из списка) спроектирована сущность Options. Для поддержки специфических типов данных (например, «Шкала» от 1 до 10 или «Сетка») была выделена вспомогательная сущность Question_Settings. Такое

разделение позволяет не перегружать основную таблицу вопросов избыточными полями, которые нужны только для определенных типов данных.

5. Сущности «Ответ респондента» (Survey_Responses и Answers)

Проектирование системы сбора данных разделено на два уровня для поддержки сессионности:

Survey_Responses фиксирует сам факт прохождения опроса конкретным пользователем в определенное время.

Answers хранит детальные ответы на каждый вопрос, ссылаясь на сессию и конкретный вопрос.

Для хранения текстов ответов было решено использовать тип данных BLOB (Binary Large Object), что позволяет пользователям давать развернутые комментарии без ограничений по длине строки.

2.2.1 Целостность данных и связи

При проектировании межтабличных связей особое внимание было уделено механизмам поддержания целостности. Были определены следующие правила:

ON DELETE CASCADE: для связей «Опрос - Вопрос» и «Вопрос - Варианты». Это гарантирует, что при удалении опроса база данных автоматически очистит все связанные с ним структуры, исключая появление «сиротских» записей.

Индексация: спроектировано создание индексов по внешним ключам и полям, участвующим в частом поиске (например, по названию опроса), что должно обеспечить высокую скорость работы системы при росте объема данных.

В результате проектирования была получена нормализованная схема данных (вплоть до третьей нормальной формы), которая позволяет эффективно хранить информацию и быстро формировать аналитические выборки. В качестве целевой системы управления базами данных была выбрана реляционная СУБД Firebird, обеспечивающая высокую производительность при работе с двоичными данными и текстовыми блоками. Финальная ER-диаграмма представлена на рисунке 5.

					МИВУ.09.03.02 025 ПЗ	Лист
						20
Изм.	Лист	№ докум.	Подпись	Дата		

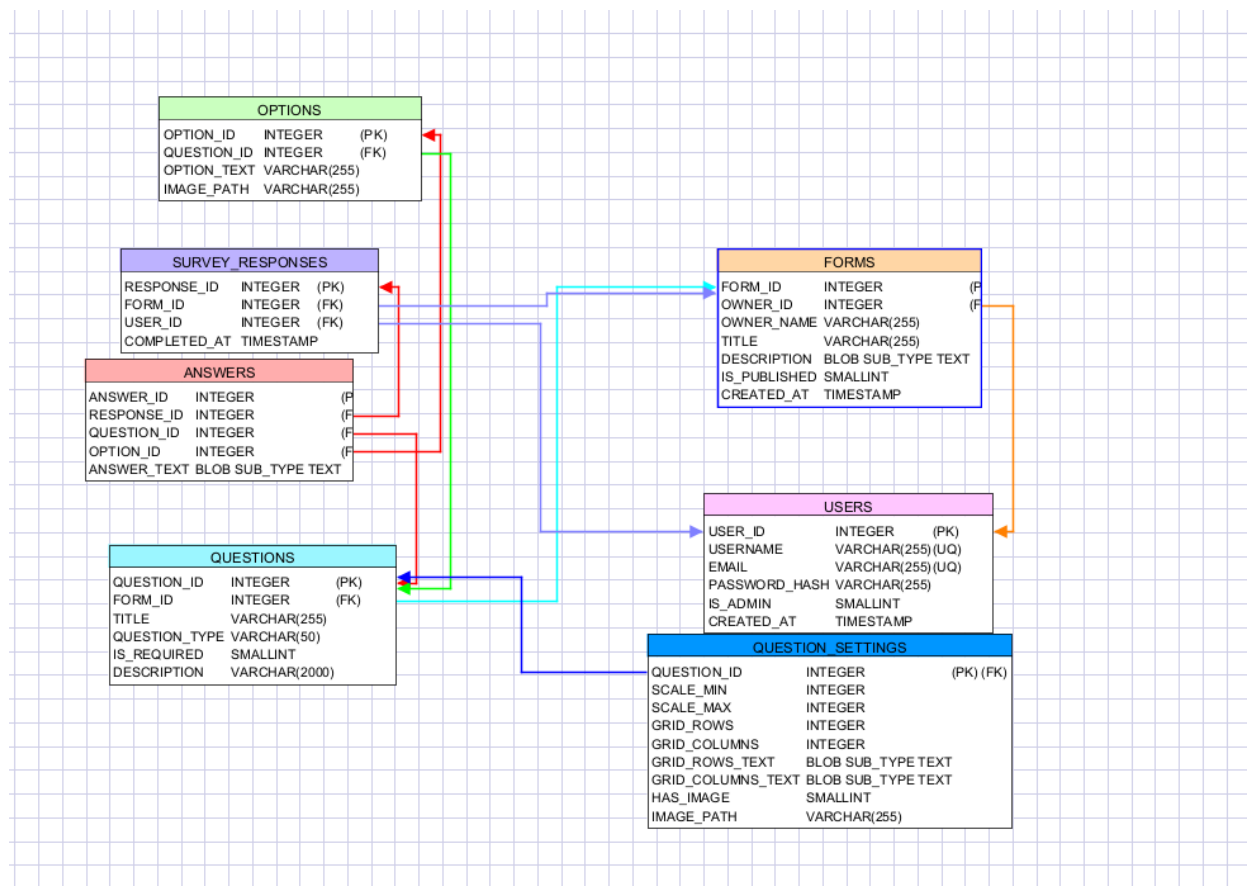


Рисунок 5– ER-диаграмма

3. Разработка приложения

При разработке проекта был реализован комплексный веб-сервис для создания, проведения и анализа онлайн-опросов на платформе Spring Boot. Сервис включает в себя полный цикл работы с опросами: от проектирования структуры анкеты и гибкой настройки различных типов вопросов до сбора ответов респондентов и формирования аналитических отчетов. Структура приложения построена по классическому паттерну MVC (Model-View-Controller), что обеспечивает четкое разделение ответственности между данными (Model), пользовательским интерфейсом (View) и управляющей логикой (Controller).

Ключевым звеном системы является слой сервисов (Service Layer), включающий компоненты FormService, QuestionService и UserService. Эти классы инкапсулируют в себе всю бизнес-логику приложения и обеспечивают высокоуровневое взаимодействие с базой данных Firebird через механизмы Spring Data JPA. Такой подход позволил абстрагироваться от низкоуровневых SQL-запросов и сосредоточиться на реализации функциональных возможностей системы, обеспечивая при этом высокую скорость разработки и чистоту кода.

3.1. Выбор и обоснование технологического стека

Для реализации серверной части приложения был выбран язык программирования Java 17 и фреймворк Spring Boot 3. Данный выбор обусловлен необходимостью создания масштабируемого и безопасного приложения с использованием современных стандартов разработки. Основными компонентами технологического стека стали:

1. Spring Data JPA и Hibernate 6.

В отличие от классического подхода с ручным управлением JDBC-

					МИВУ.09.03.02 025 ПЗ	Лист
						22
Изм.	Лист	№ докум.	Подпись	Дата		

соединениями, использование ORM-решения (Object-Relational Mapping) позволило автоматически сопоставлять Java-классы с таблицами в БД Firebird. Нюансом разработки стала настройка Hibernate Community Dialects, так как Firebird требует специфической обработки диалектов для корректной работы генерации ID и работы с типами данных BLOB.

2. Spring Security.

Для защиты пользовательских данных и разграничения прав доступа (администратор/пользователь) была внедрена подсистема безопасности. Она реализует механизмы аутентификации через сессии и обеспечивает надежное хранение паролей с использованием алгоритма хеширования BCrypt, что исключает компрометацию данных даже при прямом доступе к базе данных.

3. Thymeleaf.

В качестве движка представлений был выбран серверный шаблонизатор Thymeleaf. Его преимущество заключается в возможности создания «натуральных» шаблонов, которые остаются валидными HTML-файлами. Это упростило верстку интерфейса и позволило динамически отображать сложные структуры данных, такие как матрицы вопросов или результаты опросов в виде графиков.

4. Lombok.

Библиотека была применена для минимизации «шаблонного» кода (boilerplate). С помощью аннотаций @Data, @NoArgsConstructor и других, удалось значительно сократить объем кода в моделях данных, сделав их более читаемыми и легкими в поддержке.

5. Драйвер Jaybird.

Для обеспечения стабильного соединения с СУБД Firebird использовался нативный JDBC-драйвер Jaybird версии 5.0. Он обеспечил поддержку специфических особенностей Firebird 3.0+, включая работу с транзакциями и эффективную передачу данных.

					МИВУ.09.03.02 025 ПЗ	Лист
						23
Изм.	Лист	№ докум.	Подпись	Дата		

3.2. Реализация системы типов вопросов

Центральной особенностью разработанного сервиса является гибкая система настройки контента. В отличие от простых опросников, данное приложение поддерживает девять различных типов вопросов, что позволяет создавать как маркетинговые исследования, так и сложные технические тесты.

Реализация опирается на полиморфную структуру данных: базовые параметры (текст вопроса, обязательность, тип) хранятся в таблице QUESTIONS, а специфические настройки — в вспомогательных структурах. Для вопросов с закрытым набором ответов (MULTIPLE_CHOICE, CHECKBOX, DROPDOWN) динамически генерируются записи в таблице OPTIONS. Для наиболее сложных типов, таких как «Сетка» (Grid) или «Шкала» (Scale), используется сущность QuestionSetting, которая позволяет хранить конфигурацию интервалов или заголовков строк и столбцов в формате текстовых блоков (BLOB). На уровне пользовательского интерфейса реализована реактивная логика: при выборе типа вопроса форма динамически перестраивается, скрывая или отображая необходимые поля ввода (например, настройки минимального и максимального значения для шкалы). Реализация динамики интерфейса представлена на рисунке 6.

```
function toggleOptions() {  
  const type = document.getElementById('questionType').value;  
  const optContainer = document.getElementById('options-container');  
  const scaleSettings = document.getElementById('scale-settings');  
  const gridSettings = document.getElementById('grid-settings');  
  
  optContainer.style.display = (type === 'MULTIPLE_CHOICE' || type === 'CHECKBOX' || type === 'DROPDOWN') ? 'block' : 'none';  
  scaleSettings.style.display = (type === 'SCALE') ? 'block' : 'none';  
  gridSettings.style.display = (type === 'GRID') ? 'block' : 'none';  
}
```

Рисунок 6 – Реализация динамики интерфейса

3.3. Реализация работы с мультимедиа-контентом

Для повышения вовлеченности респондентов и возможности проведения визуальных тестов в приложении реализована подсистема работы с изображениями. Пользователь может прикрепить иллюстрацию как к самому вопросу, так и к каждому индивидуальному варианту ответа. Процесс загрузки файлов инкапсулирован в классе `FileStorageService`. Основной проблемой при разработке подобных систем является риск коллизии имен файлов (когда два пользователя загружают файлы с одинаковым названием). Данная проблема решена путем генерации уникальных идентификаторов на основе UUID, которые добавляются к оригинальному имени файла.

Файлы сохраняются в файловой системе сервера, а в базе данных фиксируется лишь относительный путь к ним. Это обеспечивает высокую производительность системы, так как база данных не перегружается тяжелыми бинарными данными, а отдает их через стандартные механизмы статических ресурсов Spring Boot. Логика хранения файлов представлена на рисунке 7.

```
public class FileStorageService {  
    public String storeFile(MultipartFile file) {  
        if (file == null || file.isEmpty()) {  
            return null;  
        }  
  
        String fileName = StringUtils.cleanPath(file.getOriginalFilename());  
        String extension = getFileExtension(fileName);  
  
        if (!allowedExtensions.contains(extension.toLowerCase())) {  
            throw new IllegalArgumentException("Недопустимый тип файла. Разрешены только: " + allowedExtensions);  
        }  
  
        try {  
            // Генерируем уникальное имя, чтобы избежать конфликтов  
            String newFileName = UUID.randomUUID().toString() + "." + extension;  
            Path targetLocation = this.fileStorageLocation.resolve(newFileName);  
            Files.copy(file.getInputStream(), targetLocation, StandardCopyOption.REPLACE_EXISTING);  
  
            return "/static/uploads/" + newFileName;  
        } catch (IOException ex) {  
            throw new RuntimeException("Could not store file " + fileName + ". Please try again!", ex);  
        }  
    }  
}
```

Рисунок 7 – Логика хранения файлов

3.4. Реализация управления жизненным циклом опроса

Управление опросами реализовано через слой контроллеров (FormController), который обеспечивает полный цикл CRUD-операций (создание, чтение, обновление, удаление). Важным аспектом является механизм разграничения прав доступа.

Система гарантирует, что операции редактирования или удаления доступны только автору опроса. Проверка осуществляется путем сопоставления идентификатора текущего авторизованного пользователя с полем owner_id в объекте формы.

Кроме того, реализована логика состояний опроса. Поле isPublished позволяет автору работать над структурой анкеты в режиме «черновика». В этом состоянии опрос не отображается в общих списках и недоступен для прохождения. После завершения настройки автор переводит опрос в опубликованное состояние, что активирует генерацию публичной ссылки и открывает доступ к сбору ответов. Логика проверки прав на опрос представлена на рисунке 8.

```
public String editForm(@PathVariable Integer id, Model model) {  
    Form form = formService.getFormById(id).orElseThrow(() -> new IllegalArgumentException(s: "Form not found"));  
    // Check ownership or admin  
    User currentUser = userDetailsService.getCurrentUser();  
    boolean isAdmin = currentUser != null && currentUser.getIsAdmin() != null && currentUser.getIsAdmin() == 1;  
    if (currentUser == null || (!form.getOwnerId().equals(currentUser.getUserId()) && !isAdmin)) {  
        return "redirect:/login";  
    }  
    model.addAttribute("form", form);  
    return "forms/edit";  
}
```

Рисунок 8 – Логика проверки прав на опрос

3.5. Реализация интерфейса прохождения и сбора данных

Процесс сбора данных является наиболее ответственным этапом работы сервиса, так как он должен корректно обрабатывать ввод от неограниченного числа респондентов и гарантировать целостность полученной информации. При переходе по уникальной ссылке на опрос система динамически формирует HTML-страницу, используя серверный шаблонизатор Thymeleaf.

Для каждого типа вопроса, хранящегося в базе данных, генерируется соответствующий ему HTML-компонент: стандартные поля ввода для текстов, переключатели radio для одиночного выбора, флажки checkbox для множественного, а также сложные динамические таблицы для сеток (Matrix вопросы).

Сбор ответов в системе реализован через специализированный компонент ResponseService. Важным архитектурным решением стало использование механизма «сессий ответов»: при каждой отправке формы в таблице SURVEY_RESPONSES создается уникальная запись, фиксирующая факт прохождения опроса и время завершения. Все индивидуальные ответы (Answers) связываются с этой записью. Это позволяет проводить комплексную аналитику: не только видеть общую статистику по вопросам, но и анализировать полный профиль ответов конкретного респондента. Для хранения данных выбран текстовый формат (BLOB/String), что позволяет унифицировать хранение: ответы на сетку сохраняются в виде структурированных пар координат (например, «строка 1 : столбец 2»), а текстовые ответы сохраняются в исходном виде. Метод сохранения ответов представлен на рисунке 9.

					МИВУ.09.03.02 025 ПЗ	Лист
						27
Изм.	Лист	№ докум.	Подпись	Дата		

```

28 public void saveResponse(Integer formId, Integer userId, Map<String, String[]> params) {
29     SurveyResponse response = new SurveyResponse();
30     response.setFormId(formId);
31     response.setUserId(userId);
32     response = responseRepository.save(response);
33
34     for (Map.Entry<String, String[]> entry : params.entrySet()) {
35         String key = entry.getKey();
36         if (key.startsWith(prefix: "q_")) {
37             Integer questionId;
38             Integer rowIndex = null;
39
40             String qPart = key.substring(beginIndex: 2);
41             if (qPart.contains(s: "_r")) {
42                 String[] parts = qPart.split(regex: "_r");
43                 questionId = Integer.parseInt(parts[0]);
44                 rowIndex = Integer.parseInt(parts[1]);
45             } else {
46                 questionId = Integer.parseInt(qPart);
47             }
48
49             String[] values = entry.getValue();
50             final Integer qId = questionId;
51             String qType = questionRepository.findById(qId).map(com.example.survey.model.Question::getQuestionType).orElse(other: "");
52
53             for (String val : values) {
54                 Answer answer = new Answer();
55                 answer.setResponseId(response.getResponseId());
56                 answer.setQuestionId(questionId);
57
58                 if (rowIndex != null) {
59                     answer.setAnswerText("row_" + rowIndex + ":" + val);
60                 } else if (qType.equals(anObject: "MULTIPLE_CHOICE") || qType.equals(anObject: "CHECKBOX") || qType.equals(anObject: "DROPDOWN")) {
61                     try {
62                         answer.setOptionId(Integer.parseInt(val));
63                     } catch (NumberFormatException e) {
64                         answer.setAnswerText(val);
65                     }
66                 } else {
67                     // For SCALE, DATE, TIME, TEXT, PARAGRAPH - always save as text
68                     answer.setAnswerText(val);
69                 }
70                 answerRepository.save(answer);
71             }
72         }
73     }
74 }

```

Рисунок 9 – Метод сохранения ответов

3.6. Реализация аналитики и экспорта результатов

Для владельца опроса в системе предусмотрен личный кабинет с

					МИВУ.09.03.02 025 ПЗ	Лист
						28
Изм.	Лист	№ докум.	Подпись	Дата		

инструментами аналитики. После того как респонденты начинают заполнять анкету, система автоматически агрегирует данные, вычисляя процентное соотношение ответов по каждому варианту и формируя списки для открытых текстовых вопросов.

Ключевым техническим решением в этом модуле стала реализация системы генерации отчетов на основе интерфейса ReportGenerator. Это позволило применить паттерн проектирования, обеспечивающий легкое расширение форматов выгрузки без изменения основного кода приложения. На данный момент реализована поддержка трех форматов:

1. HTML: Предназначен для быстрого ознакомления с результатами в интерактивном виде прямо в браузере.

2. XML: Формирует строго структурированное дерево данных, что делает его идеальным инструментом для последующего импорта в сторонние системы обработки данных или электронные таблицы.

3. TXT: Обеспечивает создание упрощенного текстового документа, удобного для печати или копирования основных тезисов.

Использование отдельных классов для каждого формата (XmlReportGenerator, TxtReportGenerator и др.) реализует принцип единственной ответственности, что упрощает отладку и поддержку кода. Метод генерации отчетов представлен на рисунке 10.

```
package com.example.survey.service;

import com.example.survey.model.Answer;
import com.example.survey.model.Form;
import com.example.survey.model.Question;
import com.example.survey.model.SurveyResponse;

import java.util.List;
import java.util.Map;

public interface ReportGenerator {
    String generateReport(Form form, List<Question> questions, List<SurveyResponse> responses, Map<Integer, List<Answer>> responseAnswers);
    String getFileType();
}
```

Рисунок 10 – Метод генерации отчетов

3.7. Архитектурные особенности реализации

При проектировании и разработке приложения особое внимание было уделено соблюдению современных архитектурных принципов, что обеспечивает гибкость, масштабируемость и легкость поддержки программного продукта. В основе системы лежат несколько ключевых концепций:

1. Многослойная архитектура (Layered Architecture).

Приложение разделено на четыре логических слоя, взаимодействие между которыми строго регламентировано:

Слой представления (Web/Controller): обрабатывает входящие HTTP-запросы, управляет навигацией и отвечает за передачу данных в шаблоны Thymeleaf.

Слой бизнес-логики (Service): содержит основную интеллектуальную составляющую приложения. Здесь сосредоточены алгоритмы обработки ответов, логика формирования отчетов и проверки прав доступа. Использование интерфейсов и сервисов позволяет изолировать бизнес-логику от деталей реализации базы данных.

Слой доступа к данным (Repository): реализован с помощью Spring Data JPA. Он абстрагирует низкоуровневые операции с СУБД Firebird, предоставляя высокоуровневый интерфейс для работы с объектами.

Слой данных (Entity): описывает структуру таблиц БД в виде Java-классов (POJO), что позволяет использовать все преимущества объектно-ориентированного подхода при работе с информацией.

2. Управление транзакциями.

Для обеспечения целостности данных при выполнении сложных операций (например, одновременное сохранение опроса и всех его вопросов) в слое сервисов активно применяется аннотация `@Transactional`. Это гарантирует, что операция будет выполнена либо полностью, либо не выполнена вовсе, предотвращая появление «частично сохраненных» или поврежденных данных в базе Firebird.

					МИВУ.09.03.02 025 ПЗ	Лист
						30
Изм.	Лист	№ докум.	Подпись	Дата		

3. Инверсия управления (IoC) и внедрение зависимостей (DI).

Использование механизмов Spring Framework для управления жизненным циклом компонентов позволило добиться слабой связанности (low coupling) между классами. Компоненты системы не создают зависимости вручную, а получают их через конструкторы, что значительно упрощает модульное тестирование и замену отдельных частей системы.

4. Безопасность как сквозная функциональность.

Архитектура безопасности интегрирована на уровне фильтров запросов. Благодаря Spring Security, проверка прав доступа осуществляется до того, как запрос попадает в контроллер. Это позволяет централизованно управлять доступом и защищать конфиденциальные данные пользователей без дублирования логики проверок в каждом методе.

					МИВУ.09.03.02 025 ПЗ	Лист
						31
Изм.	Лист	№ докум.	Подпись	Дата		

4. Тестирование

Одним из важнейших этапов создания программного продукта является его тестирование и отладка. Тестирование позволяет выявить скрытые и явные недостатки программы, либо убедиться в ее пригодности для применения.

Целью тестирования является проверка работоспособности программы, то есть правильности выполнения всех функций, описанных выше.

Система включает многоуровневую валидацию для предотвращения некорректных операций. Были протестированы следующие сценарии отображенные в таблице 3, результат тестов показан на рисунках 11-18.

Таблица 3 – Сценарии тестирования

№	Модуль	Действие	Ожидаемый результат	Статус
1.	Аутентификация	Регистрация нового пользователя	Создается учетная запись, происходит редирект на страницу входа	Выполнено
2.	Аутентификация	Регистрация нового пользователя с неполными данными	учетная запись не создается, указывается на поля которые необходимо заполнить	Выполнено
3.	Аутентификация	Вход с корректными данными	Устанавливается сессия, редирект на список форм	Выполнено
4.	Аутентификация	Вход с неверным паролем/логином	Отображение сообщения об ошибке	Выполнено
5.	Аутентификация	Выход из системы	Удаление сессии, редирект на главную	Выполнено
6.	Управление формами	Создание новой формы	Форма появляется в списке "Мои формы"	Выполнено
7.	Управление формами	Удаление формы	Форма исчезает из списка	Выполнено
8.	Управление формами	Публикация формы (статус "public")	Форма появляется в публичном каталоге	Выполнено
9.	Управление формами	Публикация формы (статус "link")	Генерируется уникальная ссылка	Выполнено
10.	Управление вопросами	Добавление вопросов разного типа	Все вопросы корректно создаются	Выполнено
11.	Управление вопросами	Редактирование вопросов	Успешно редактируются поля с вариантами ответов	Выполнено

Продолжение таблицы 3 – Сценарии тестирования

№	Модуль	Действие	Ожидаемый результат	Статус
12.	Управление вопросами	Добавление изображения к вопросу	Изображение отображается при прохождении	Выполнено
13.	Прохождение опросов	Доступ к публичной форме	Форма отображается для заполнения	Выполнено
14.	Прохождение опросов	Доступ к форме по ссылке	Форма доступна только с токеном	Выполнено
15.	Прохождение опросов	Попытка доступа к неопубликованной форме	Отображение ошибки 403 (Forbidden)	Выполнено
16.	Прохождение опросов	Отправка пустого обязательного вопроса	Валидация на стороне сервера, отображение ошибки	Выполнено
17.	Анализ ответов	Просмотр списка ответов на форму	Таблица с ответами пользователей	Выполнено
18.	Анализ ответов	Просмотр деталей конкретного ответа	Ответы в контексте вопросов	Выполнено
19.	Анализ ответов	Просмотр статистики	Диаграммы и графики по ответам	Выполнено
20.	Анализ ответов	Экспорт в TXT	Скачивание .txt файла	Выполнено
21.	Фильтрация	Поиск форм по названию	Отображение только соответствующих форм	Выполнено

1. Регистрация с некорректными данными:

Регистрация

Имя пользователя

Email

! Адрес электронной почты должен содержать символ "@". В адресе "11" отсутствует символ "@".

...

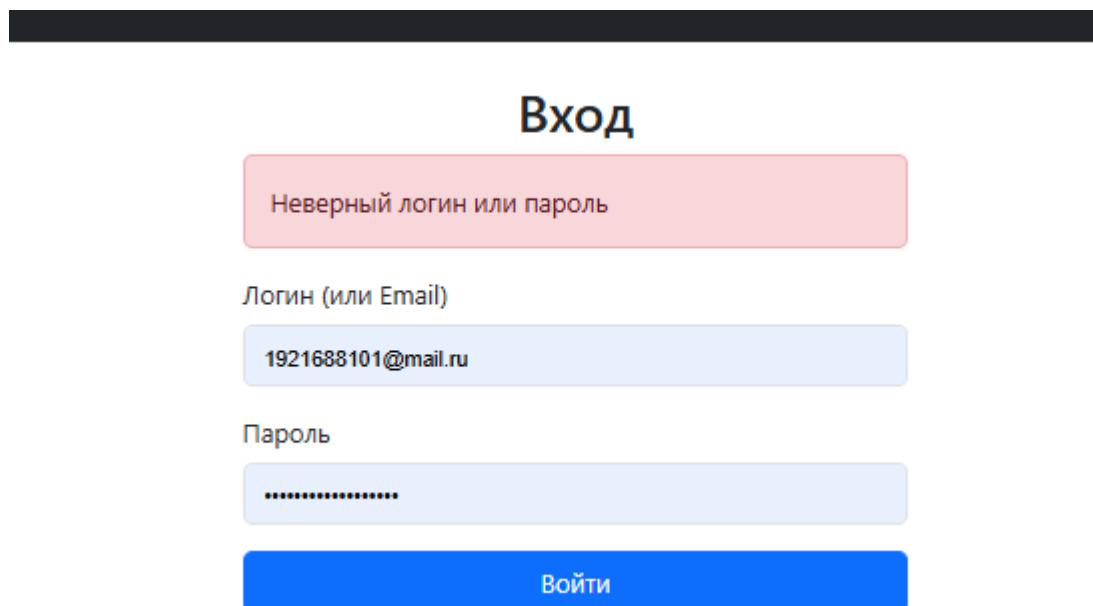
Подтвердите пароль

Зарегистрироваться

[Уже есть аккаунт? Войти](#)

Рисунок 11 – попытка регистрации с некорректными данными

2. Авторизация с некорректными данными:



Вход

Неверный логин или пароль

Логин (или Email)

1921688101@mail.ru

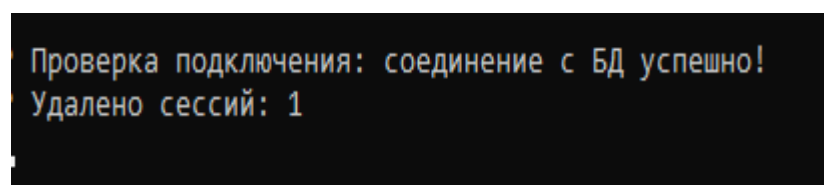
Пароль

.....

Войти

Рисунок 12 – попытка авторизации с некорректными данными

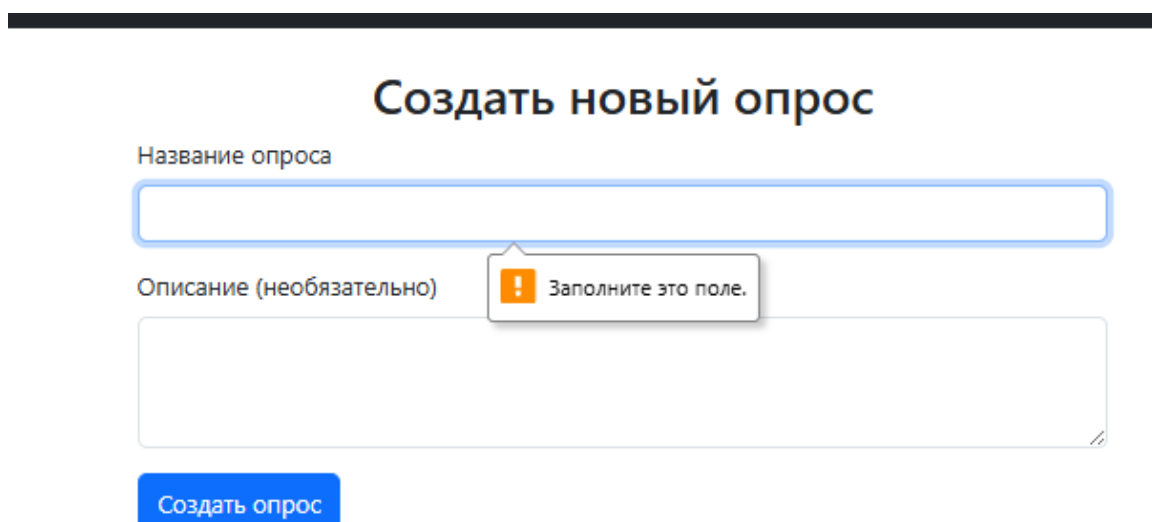
3. Выход из системы:



Проверка подключения: соединение с БД успешно!
Удалено сессий: 1

Рисунок 13 – Выход из системы

4. Создание формы без названия:



Создать новый опрос

Название опроса

Описание (необязательно)

Заполните это поле.

Создать опрос

Рисунок 14 – Создание формы без названия

5. Загрузка неграфического файла:

```
2026-05-13T11:11:47.639+03:00 ERROR 21732 --- [JavaSurveyApp] [nio-8080-exec-9] o.a.c.c.C.[.][.].dispatcherServlet : Servlet.service() for servlet [dispatcherServlet] in context with path [] threw exception [Request processing failed: java.lang.IllegalArgumentException: =>хфояеёёёшъуц ёшя Ырщыр. ЪрчЕх°хэв Ёшь№ью: [jpg, jpeg, png, gif]] with root cause
java.lang.IllegalArgumentException: =>хфояеёёёшъуц ёшя Ырщыр. ЪрчЕх°хэв Ёшь№ью: [jpg, jpeg, png, gif]
    at com.example.survey.service.FileStorageService.storeFile(FileStorageService.java:43) ~[classes/:na]
    at com.example.survey.controller.QuestionController.create(QuestionController.java:56) ~[classes/:na]
    at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method) ~[na:na]
```

Рисунок 15 – Загрузка неграфического файла

Так как файл не будет иметь стандартного расширения, то и отображаться он не будет. Так же на стороне клиента будет выведено сообщение об ошибке.

6. Создание опроса и публикация

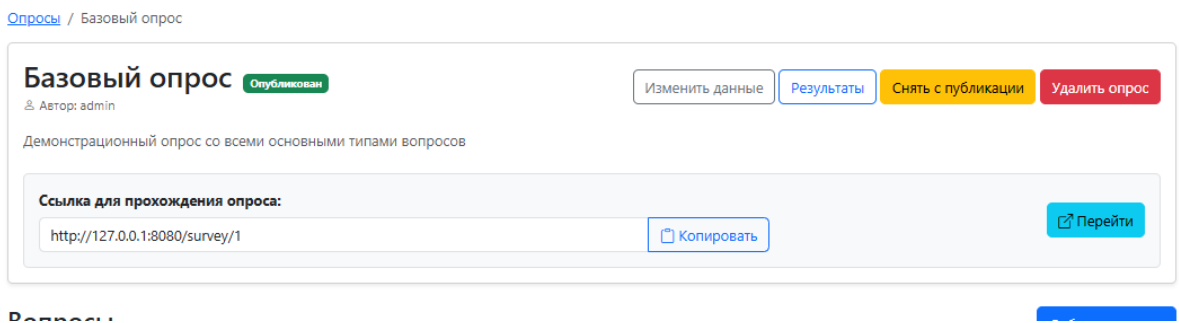


Рисунок 16 – публикация формы

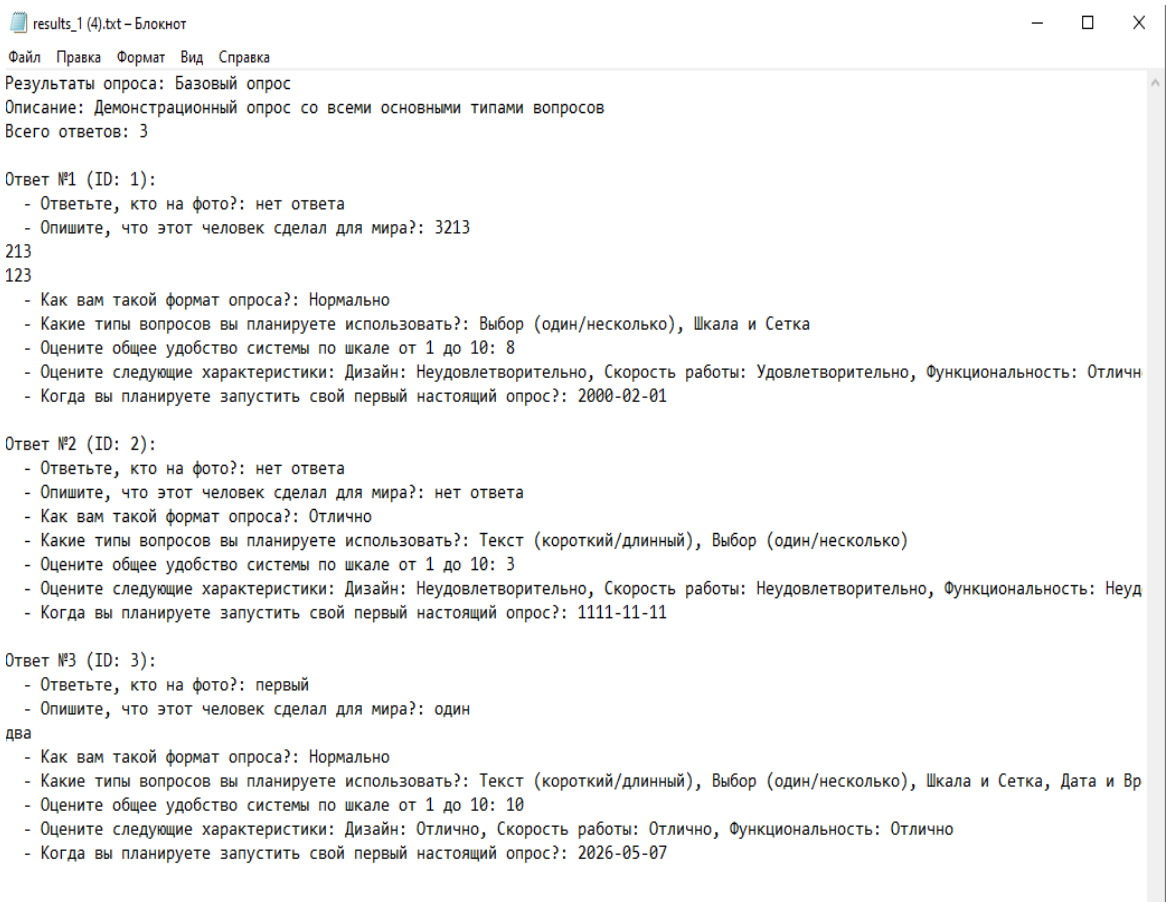


Рисунок 17 – Скачивание результатов в формат .txt

3. Как вам такой формат опроса?

Вариант	Кол-во	%
Хорошо	0	0%
Отлично	1	33%
Плохо	0	0%
Нормально	2	66%

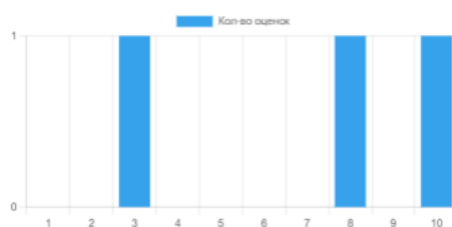


4. Какие типы вопросов вы планируете использовать?

Вариант	Кол-во	%
Дата и Время	1	33%
Выбор (один/несколько)	3	100%
Шкала и Сетка	2	66%
Текст (короткий/длинный)	2	66%



5. Оцените общее удобство системы по шкале от 1 до 10



6. Оцените следующие характеристики

	Неудовлетворительно	Удовлетворительно	Хорошо	Отлично
Дизайн	2	0	0	1

Рисунок 18 – Просмотр статистики ответов

Заключение

В ходе выполнения курсовой работы был разработан веб-сервис для создания, проведения и анализа онлайн-опросов на языке Java. Основной целью проекта являлось создание многофункционального приложения, способного гибко настраивать структуру анкет с различными типами данных, собирать ответы респондентов и предоставлять удобные инструменты для аналитики полученных результатов. В процессе разработки была спроектирована классическая многослойная архитектура на основе паттерна MVC (Model-View-Controller), включающая подсистемы аутентификации, управления формами опросов, настройки вопросов, сбора ответов и генерации отчетов. Такой подход позволил четко разделить ответственность между бизнес-логикой, взаимодействием с базой данных и пользовательским интерфейсом, обеспечив независимость компонентов системы.

В рамках работы была реализована поддержка девяти типов вопросов, включая сложные структуры, такие как шкалы оценок и матричные сетки (Grid). Для управления бизнес-процессами использовались классы сервисного слоя (FormService, QuestionService, ResponseService), а взаимодействие с хранилищем данных осуществлялось через интерфейсы Spring Data JPA. Вся логика валидации, сохранения ответов и расчета статистики выполняется на сервере, что гарантирует целостность данных при одновременном прохождении опросов множеством пользователей. Для детального анализа собранных результатов была разработана подсистема агрегации данных и генерации отчетов. Она поддерживает экспорт статистики в форматы HTML, XML и TXT. Дополнительно была реализована функциональность работы с мультимедиа, позволяющая безопасно загружать и прикреплять изображения к анкетам. При разработке активно применялся фреймворк Spring Boot, а также Spring Security для реализации ролевой модели доступа и хеширования паролей. Пользовательский интерфейс построен с использованием серверного шаблонизатора Thymeleaf. В качестве системы

					МИВУ.09.03.02 025 ПЗ	Лист
						37
Изм.	Лист	№ докум.	Подпись	Дата		

управления базами данных была выбрана СУБД Firebird. Для обеспечения расширяемости подсистемы экспорта отчетов был успешно применен паттерн проектирования Strategy.

В результате выполнения курсовой работы была создана полнофункциональная и масштабируемая система проведения онлайн-опросов, соответствующая всем поставленным требованиям. Разработанный продукт наглядно демонстрирует применение принципов объектно-ориентированного проектирования и современных стандартов промышленной веб-разработки на платформе Java.

					МИВУ.09.03.02 025 ПЗ	Лист
						38
Изм.	Лист	№ докум.	Подпись	Дата		

Библиография

Ермаков, А. В. Разработка приложений на языке JAVA : учебное пособие / А. В. Ермаков, М. С. Королёв, А. К. Кузьмин. — Саратов : Саратовский государственный технический университет имени Ю.А. Гагарина, ЭБС АСВ, 2024. — 128 с. — ISBN 978-5-7433-3635-7. — Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. — URL: <https://www.iprbookshop.ru/150088.html>

Системы управления базами данных [Электронный ресурс]: лабораторный практикум/ – Электрон текстовые данные. – Ставрополь: Северо-Кавказский федеральный университет, 2017. – 148 с. – Режим доступа: <http://www.iprbookshop.ru/75595.html>. – ЭБС «IPRbooks»

Лазицкас Е.А. Базы данных и системы управления базами данных [Электронный ресурс]: учебное пособие/ Лазицкас Е.А., Загумённикова И.Н., Гилевский П.Г. – Электрон. текстовые данные. – Минск: Республиканский институт профессионального образования (РИПО), 2018. – 268 с. – Режим доступа: <http://www.iprbookshop.ru/93382.html>. – ЭБС «IPRbooks»

Мирошников А.И. Архитектура систем управления базами данных [Электронный ресурс]: учебное пособие/ Мирошников А.И. – Электрон. текстовые данные. – Липецк: Липецкий государственный технический университет, ЭБС АСВ, 2018. – 94 с. – Режим доступа: <http://www.iprbookshop.ru/83189.html>. – ЭБС «IPRbooks»

Николаев Е.И. Базы данных в высокопроизводительных информационных системах [Электронный ресурс]: учебное пособие/ Николаев Е.И. – Электрон. текстовые данные.— Ставрополь: Северо-Кавказский федеральный университет, 2016. – 163 с. – Режим доступа: <http://www.iprbookshop.ru/69375.html>. – ЭБС «IPRbooks»

					МИВУ.09.03.02 025 ПЗ	Лист
						39
Изм.	Лист	№ докум.	Подпись	Дата		

Яндекс – Документация по Яндекс.Формам. – URL:
<https://yandex.ru/support/forms/>

Google LLC – Помощь по Google Forms. — URL:
<https://support.google.com/docs/answer/6281888>

					МИВУ.09.03.02 025 ПЗ	Лист
						40
Изм.	Лист	№ докум.	Подпись	Дата		